



Novedades de la versión 1.4

Solo lo que cambió en la API de integración: integraciones nuevas, modificaciones y correcciones. Compatible con lo que ya tienen integrado: sigue funcionando sin tocarlo.

Fecha 19 de septiembre de 2026 **Aplica sobre** documento de integración v1.3 (31 de julio de 2026)

Emite Puntos Plus – Gasolineras Turkaj, Chichicastenango

Este documento contiene **únicamente lo que cambió** respecto a la v1.3, a partir de sus respuestas a los apartados 10 y 11. El documento completo (v1.4) sigue siendo la referencia general; acá está solo lo nuevo.

Compatible con lo que ya integraron. Ningún campo existente cambió de nombre, tipo ni significado, y los campos nuevos son opcionales. El único requisito nuevo es que `invoice_no` pasa a ser obligatorio (3.1) — y ya lo envían en todas sus llamadas, así que **no tienen que cambiar nada para seguir funcionando.**

Resumen

#	CAMBIO	TIPO	QUÉ DEBE HACER PROPER
2.1	<code>operator.dpi</code> — DPI del colaborador	Integración nueva	Enviar el DPI en <code>operator</code> (recomendado)
2.2	<code>reward_value</code> — valor monetario del premio	Integración nueva (solicitada por PROPER)	Leer el campo; tolerar <code>null</code>
2.3	<code>expires_at</code> — vencimiento del premio	Integración nueva	Opcional: mostrarlo; tolerar <code>null</code>
3.1	Acumulación solo al momento de la factura + factura única	Regla de negocio	Mostrar el botón al emitir la factura; manejar <code>409 invoice_already_credited</code>
3.2	Códigos de estación configurados	Modificación	Enviar su código en <code>operator.station</code>
3.3	Varios combustibles en una factura	Criterio confirmado	Nada — se mantiene lo que ya hacen
3.4	Nombre real del colaborador	Modificación	Enviar el nombre real en <code>operator.name</code>
3.5	<code>total_amount</code> = total de la factura	Aclaración	Enviar el total con tienda cuando aplique
4.1	Error <code>expired</code>	Corrección	Manejar <code>422 expired</code> en canjes
4.2	Tarjetas no activas	Corrección	Nada — mismo error <code>member_not_found</code>
4.3	Códigos de ejemplo vs. códigos de prueba	Aclaración	Usar los códigos del apartado 8

02

Integraciones nuevas

2.1 `operator.dpi` — el DPI como identificador estable del colaborador

Por qué. Nos indicaron que el identificador interno del usuario puede variar si al colaborador se le crea otro usuario en una sucursal distinta, y propusieron el DPI como dato en común. Lo adoptamos.

Qué cambia. El objeto `operator` acepta un campo nuevo, **opcional**:

```
"operator": {
  "external_id": "351",
  "name": "Juan Pérez",
  "dpi": "2990123450101",
  "station": "17261015-1"
}
```

Aplica en los dos lugares donde viaja `operator`: `POST /v1/purchases` y `POST /v1/redemptions` (acción `deliver`).

Cómo lo usamos.

SITUACIÓN	RESULTADO
<code>external_id</code> ya conocido	Se usa ese colaborador (como hasta hoy). Si aún no teníamos su DPI, lo guardamos
<code>external_id</code> nuevo + DPI que ya conocemos	Se une al mismo colaborador : conserva su historial de atención y calificaciones
<code>external_id</code> nuevo + DPI desconocido (o sin DPI)	Se crea el colaborador, como hasta hoy

Reglas del campo.

- 13 dígitos. Se acepta con espacios o guiones (`2990 12345 0101`): los quitamos de nuestro lado.
- Un DPI con otro largo **se ignora**: la compra o la entrega se procesa igual, solo que sin esa unión.
- `external_id` **sigue siendo obligatorio** — el DPI lo complementa, no lo reemplaza.
- El DPI **nunca** se devuelve en ninguna respuesta y en nuestra bitácora de llamadas se guarda enmascarado (solo los últimos 4 dígitos).

2.2 `reward_value` — valor monetario del premio

Por qué. Solicitaron que, al consultar un canje, el JSON incluya el valor monetario al que equivale el premio.

Qué cambia. Todas las respuestas de `/v1/redemptions` incluyen ahora `reward_value`, en **quetzales**:

GET /v1/redemptions?code=TK-...	En la raíz de la respuesta
GET /v1/redemptions?card_code=CT...	En cada elemento de <code>pending[]</code>
POST /v1/redemptions · <code>action: request</code>	En la raíz de la respuesta
POST /v1/redemptions · <code>action: deliver</code>	En la raíz (payload del comprobante)

```
{
  "ok": true,
  "redemption_id": "4b2c...",
  "code": "TK-54C5E8",
  "reward_name": "Vale Q10 Combustible",
  "category": "combustible",
  "reward_value": 10.00,
  "points_spent": 80,
  "member_name": "Cliente Prueba 1",
  "card_code": "CTPD-95176",
  "created_at": "2026-07-31T19:03:04.640Z",
  "expires_at": null,
  "delivered": false,
  "delivered_at": null,
  "confirm_status": "none"
}
```

A tener en cuenta.

- Tipo `number` con hasta 2 decimales, o `null`. Es `null` cuando el premio no tiene un valor monetario definido (por ejemplo, un artículo promocional): el POS debe tolerarlo y no asumir cero.
- Es el **valor del premio**, no un monto cobrado: los premios se pagan con puntos (`points_spent`) y el cliente no paga nada al recibirlos.
- Los premios obtenidos por promoción o por rifa (`points_spent: 0`) también traen su valor cuando está definido.

2.3 `expires_at` — fecha límite para reclamar el premio

Las dos consultas `GET /v1/redemptions` incluyen ahora `expires_at`:

- `null` → el premio **no vence** (el caso normal de los canjes).

- Fecha ISO 8601 UTC → fecha límite. Hoy solo la llevan los **premios de rifa**. Pasada esa fecha, `request` y `deliver` responden `422 expired` (ver 4.1).

Sirve para avisarle al colaborador **antes** de iniciar la entrega, en lugar de descubrirlo con el error.

03

Modificaciones y criterios confirmados

3.1 Acumulación solo al momento de la factura — y factura única

Respuesta a su pregunta sobre el cliente sin tarjeta escaneada: no necesitan parametrizar nada, porque **no habrá acumulación posterior**. La regla del programa es:

- Al emitir la factura, el POS muestra **en ese instante** el botón para escanear el QR del cliente y asignar los puntos.
- Si el colaborador no lo hace y **emite otra factura**, los puntos de la factura anterior **quedan sin asignar**. No hay forma de acumular una factura ya cerrada.

Del lado de la API respaldamos la regla con un **candado de factura única**:

SITUACIÓN	RESPUESTA
Compra sin <code>invoice_no</code>	<code>422 missing_invoice_no</code> — el número de factura pasa a ser obligatorio
<code>invoice_no</code> que ya acreditó puntos	<code>409 invoice_already_credited</code> — no se acredita de nuevo, aunque llegue con otra tarjeta, otra estación u otra <code>Idempotency-Key</code>

```
{
  "error": "invoice_already_credited",
  "message": "Esta factura ya acreditó puntos a esta tarjeta",
  "invoice_no": "FC351-22",
  "same_card": true,
  "credited_at": "2026-09-18T19:18:06.510Z",
  "purchase_id": "9c1e...",
  "points_earned": 2
}
```

- `same_card: true` → la acreditación original fue a **esa misma tarjeta**; incluimos `purchase_id` y `points_earned` para que el POS pueda mostrarlos (caso típico: un reintento sin

Idempotency-Key). Pueden tratarlo como "ya acumulada".

- `same_card: false` → la factura ya se acreditó a **otra** tarjeta; no revelamos a quién.
- La comparación del número ignora mayúsculas/minúsculas y espacios al inicio y al final. El número debe ser único por factura (serie-número).
- **La idempotencia no cambia:** un reintento con la **misma** Idempotency-Key sigue devolviendo la respuesta original con `"replayed": true`. El candado actúa cuando la llave es distinta o no viene.

3.2 Códigos de estación — ya configurados

Cargamos de nuestro lado los códigos que nos entregaron:

OPERATOR.STATION	ESTACIÓN EN PUNTOS PLUS
17261015-1	Turkaj I
17261015-2	Turkaj II
105978272-3	Turkaj III (Estación de Servicio La Cruz)

En sus pruebas vimos que resolvieron la estación enviando nuestro identificador interno (03643c23-...). **Eso sigue funcionando**, pero ya pueden enviar directamente su propio código, que es lo previsto en el contrato. Un código que no esté en esta tabla responde `422 unknown_station`; si abren una sucursal nueva, avísennos para agregarla.

3.3 Varios combustibles en una misma factura — se mantiene su criterio

Confirmado tal como lo tienen: `fuel_amount`, `gallons` y `total_amount` van **sumados**, y en `fuel_type` el **primer combustible de la lista** (los productos que no son combustible quedan fuera de `fuel_amount`). No necesitamos un arreglo de líneas. Los puntos se calculan sobre `fuel_amount`, así que no se ven afectados.

De nuestro lado, la instrucción a los colaboradores será emitir **una factura por producto**, por lo que este caso debería ser excepcional.

3.4 Nombre real del colaborador

En las pruebas recibimos `operator.name = "PROPER"` para todas las compras. En producción necesitamos el **nombre real** de quien atendió: es el que ve el cliente en su notificación ("Atendido por Juan") y al calificar la atención. Solo lo tomamos la primera vez que aparece el colaborador; después respetamos el nombre que tengamos registrado.

3.5 `total_amount` es el total de la factura

En las pruebas `total_amount` llegó siempre igual a `fuel_amount`. El criterio es:

- `fuel_amount` → suma de **combustibles** (base de los puntos). ✓ Ya lo envían así.
- `total_amount` → **total facturado**, incluyendo tienda, lubricantes, etc.

No afecta los puntos; solo lo guardamos para conciliar. Si la factura es solo de combustible, ambos valores coinciden y está bien.

3.6 Combustible por bomba — cerrado

Entendido: el producto es general y el número de bomba es un parámetro aparte. No lo necesitamos; `fuel_type` sigue siendo `super`, `regular` o `diesel`.

04

Correcciones

4.1 Error `expired` — ahora documentado y con su código HTTP correcto

Un premio con plazo vencido ya era rechazado, pero el error no figuraba en la tabla del apartado 6 y respondía con HTTP 400. Desde esta versión:

HTTP	ERROR	CUÁNDO	CAMPOS EXTRA
422	<code>expired</code>	<code>request</code> o <code>deliver</code> sobre un premio cuyo <code>expires_at</code> ya pasó	<code>expired_at</code> (ISO 8601)

```
{
  "error": "expired",
  "message": "El plazo para reclamar este premio venció el 15/09/2026",
  "expired_at": "2026-09-15T18:00:00.000Z"
}
```

`cancel` sí se acepta sobre un premio vencido (para cerrar una solicitud que hubiera quedado abierta).

4.2 Tarjetas no activas — mismo comportamiento en todos los endpoints

Una tarjeta bloqueada o inactiva ya no identificaba al cliente en `GET /v1/members`. Ahora el criterio es el mismo en `POST /v1/purchases` y en `GET /v1/redemptions?card_code=`: responden `404 member_not_found`. **No hay códigos de error nuevos.**

4.3 Códigos de ejemplo y parámetros de consulta

Dos observaciones de la bitácora de sus pruebas, por si ayudan:

- `TK-3F9A2C` y `CTOD-00042` son **ejemplos ilustrativos** del documento y no existen (responden 404). Los códigos reales de prueba son los del apartado 8: tarjetas `CTPD-95176` (antes `CTOD-95176`: Cliente Prueba 1 subió a PLATINO durante sus pruebas, y con el nivel cambia la letra de la tarjeta) y `CTOD-93935`, y la lista viva de canjes se obtiene con `GET /v1/redemptions?card_code=CTPD-95176`.
- `GET /v1/members` y `GET /v1/redemptions` leen sus parámetros de la **query string** (`?card_code=...` o `?code=...`). Sin parámetro responden `400 invalid_card_code` / `404 redemption_not_found`.

La colección de Postman actualizada (`PuntosPlus-PROPER.postman_collection.json`, v1.4) ya incluye `operator.dpi` y el código de estación en sus variables.

Documento de novedades. Complementa al documento de integración v1.4, que sigue siendo la referencia completa. Cualquier duda la resolvemos por el canal que les resulte más cómodo.